

Video Processing — Nightly Exam Recap

0512-4263 • Prof. Shai Avidan, TAU • every algorithm: what goes in, what comes out, what the knobs do

READ THIS PAGE FIRST — it is worth more than the rest

The exam is always 8 questions, one per slot, 12–13 pts each. Name the slot $\Rightarrow$ you know the chapter $\Rightarrow$ you know the machinery.

#	Slot	Ch	What it always asks
1	Optic flow	2	“Modify Horn–Schunck to also use extra input X ” (labels, depth, confidence, edges)
2	Tracking	4	Particle-filter resampling (inverse-CDF), or derive the Bayes recursion
3	Intrinsic / dyn. texture	12 / 5	Weiss median made <i>online</i> , or state-space synthesis in a new domain
4	“Paper” slot	—	A paper shown in class (time fronts, light fields). Often outside the notes
5	PatchMatch / retargeting	11	Bend the NN field: clone detection, k -NN, inpainting, video
6	Geodesic distance	9/10	Distance map or shortest path; edge- <i>avoiding</i> vs edge- <i>following</i> ; γ
7	Segmentation	9	Propagate seed labels by geodesic distance on some graph
8	Clustering / theory	6/11	“Is it guaranteed / symmetric?” $\Rightarrow$ monotone proof or counter-example

Four recipes that solve nearly every question:

1. “**Modify A to accept new input B.**” Ask one thing: is B a *boundary signal* or a *measurement*? Boundary $\Rightarrow$ put it in the *smoothness* weight (turn smoothing off across it). Measurement $\Rightarrow$ put it in the *data* term (anchor / seed / confidence weight).
2. “**Make it online / efficient.**” Replace “store everything” by a running statistic: per-pixel 256-bin histogram (running median/mode), incremental Gaussian update, low-rank factorisation, PatchMatch instead of brute-force NN.
3. “**Adapt an image method to a new domain**” (3D points, social graph, words, notes): build the right graph or vector space first, then reuse the *same* machinery — Dijkstra geodesics, ARMA state space, or NN fields.
4. “**Is X guaranteed / symmetric / a metric?**” Either a monotone-decrease + finiteness argument (terminates), or a 2–3 point counter-example (not symmetric).

The one hammer: define a cost $\rightarrow$ add a boundary-aware regulariser $\rightarrow$ minimise iteratively (least squares / Dijkstra-DP / EM-style alternation).

1 Ch.1 — Motion Representation

The transformation ladder: linear $\rightarrow$ affine $\rightarrow$ homography $\rightarrow$ projection

slot: background for all

IN pairs of corresponding points (p_i, p'_i) between two images (or a 3D point + a calibrated camera).

OUT the matrix that maps every pixel of image 1 onto image 2.

Flow (in words)

1. Pick the model = how much freedom the motion needs.
2. Each point pair gives **2 equations** (one for x' , one for y').
3. A model with k free parameters needs $\lceil k/2 \rceil$ pairs. More pairs $\Rightarrow$ over-determined $\Rightarrow$ least squares.

Math

Model	Form	Free params	Min. pairs / keeps
Linear 2×2	$p' = Ap$	4	2 pairs; <i>cannot translate</i> (origin is fixed)
Affine 2×3	$p' = (A \ t) (x, y, 1)^T$	6	3 pairs; parallel lines stay parallel
Homography 3×3	$p' \cong Hp$, divide by 3rd coord	8 (9 up to scale)	4 pairs; a plane under perspective
Projection 3×4	$p \cong K \Pi [R \ t] (X, Y, Z, 1)^T$	11 (6 ext. + 5 int.)	full 3D $\rightarrow$ pixel

Least squares for all of them: $\tilde{A}\tilde{x} = b \Rightarrow \tilde{x} = (\tilde{A}^T \tilde{A})^{-1} \tilde{A}^T b$.

where: A = rotation/scale/shear block; $t = (t_x, t_y)$ = translation, only reachable thanks to the homogeneous “1”; R, t = *extrinsics* (where the camera is and points); K with f, k_u, k_v, u_0, v_0 = *intrinsics* (rays $\rightarrow$ pixels); the $1/Z$ inside Π is what makes far things small.

Knobs

- **model richness** $\uparrow$ — fits more complex motion but is far more noise-sensitive and needs more correspondences.
- **# point pairs** $\uparrow$ — LS averages out noise; fewer pairs than needed $\Rightarrow$ system unsolvable.
- **normalisation of H** — H is only defined up to scale; without fixing it the solve is degenerate.

Exam trap a homography is *not* affine: parallel lines may converge. And 2×2 can never translate — if the answer needs a shift, you need the homogeneous coordinate.

2 Ch.2 — Optical Flow

Everything here sits on the **brightness constancy** assumption: a moving point keeps its intensity, $I_1(x, y) = I_2(x+u, y+v)$. Taylor $\Rightarrow I_x u + I_y v + I_t = 0$: *one equation, two unknowns*. Every method below is a different way to add the missing equation.

Lucas–Kanade (LK)

Ch.2 • sparse flow

IN two frames I_1, I_2 ; a window W around a pixel; (optionally) a warp model.

OUT one motion vector (u, v) per window — or the 6 affine warp parameters P .

Flow (in words)

1. **Missing equation** $\rightarrow$ assume every pixel in the window shares *one* motion. Now n^2 equations, 2 unknowns.

2. Build the over-determined system and solve it by least squares (a 2×2 inverse — very cheap).
3. Because Taylor is only valid for small motion, **iterate**: warp I_2 by the current estimate, re-measure the residual, add the correction.
4. For big motion, run steps 1–3 on an image **pyramid**, coarse to fine.

Math

$$x = (A^T A)^{-1} A^T b, \quad A^T A = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix}, \quad b = -\sum I_t$$

where: I_x, I_y = spatial gradients (how fast brightness changes sideways — the “lever arm” that converts motion into intensity change); I_t = temporal difference $I_2 - I_1$ (how much the pixel actually changed); $A^T A$ = the *structure tensor*, summed over the window — it says whether the window has enough texture to pin down 2 unknowns; the sum is what turns 1 equation per pixel into a solvable system.

General warp: $P^* = \arg \min_P \sum_X [I_2(W(X; P)) - I_1(X)]^2$, updated by $\Delta P^* = -(B^T B)^{-1} B^T I_t$ with $B = \nabla I_2 \partial W / \partial P$.

Knobs

- **window size** $\uparrow$ — more equations $\Rightarrow$ more robust to noise, but mixes two different motions if the window straddles an object boundary. $\downarrow$: worse aperture problem, rank-deficient.
- **warp model** — translation (2 params) is stable on small windows; affine (6) fits large windows but needs much more texture.
- **pyramid levels** $\uparrow$ — handles larger motion (each level halves the displacement), but coarse levels lose small objects.

Exam trap LK fails exactly when $A^T A$ is singular: flat region (rank 0) or a straight edge (rank 1 = **aperture problem**, only the normal component is observable). That is the whole reason Harris exists.

Harris corner detector

Ch.2 • where LK is allowed to run

IN one image.

OUT a list of corner keypoints = pixels where flow is fully determined.

Flow (in words)

1. Take the *same* matrix LK already builds, $H = A^T A$ (structure tensor), on a window around every pixel.
2. Its two eigenvalues say what the window looks like: both small = flat, one large = edge, both large = corner.
3. Compress the two eigenvalues into one score R so you never have to do an eigendecomposition.
4. Threshold R , then non-maximum suppression so corners are strong *and* spread out.

Math

$$E(u, v) = (u \ v) H \begin{pmatrix} u \\ v \end{pmatrix}, \quad R = \underbrace{\det H - \alpha(\text{tr } H)^2}_{\text{Harris}} \quad \text{or} \quad \underbrace{\lambda_{\min}}_{\text{Shi-Tomasi}} \quad \text{or} \quad \underbrace{\det H / \text{tr } H}_{\text{Noble}}$$

where: $E(u, v)$ = how much the window changes if you shift it by (u, v) (big change in *all* directions = corner); $\det H = \lambda_{\min} \lambda_{\max}$ = “are both directions strong?”; $\text{tr } H = \lambda_{\min} + \lambda_{\max}$ = total gradient energy; α subtracts a penalty that kills the “one huge eigenvalue” (edge) case.

Knobs

- $\alpha \approx 0.04$ – 0.06 $\uparrow$ — rejects more edges, keeps only sharp corners. $\downarrow$: edges start passing as corners.
- **threshold** θ $\uparrow$ — fewer but stronger keypoints (good for RANSAC), risk of too few to fit a model.
- **window size** $\uparrow$ — smoother, more rotation-robust H , but poorer localisation.

Exam trap Harris is *not* scale-invariant — a corner at one zoom is an edge at another. Pair it with a pyramid.

Horn–Schunck (HS)

Ch.2 • dense flow • exam slot 1

IN two frames I_1, I_2 ; smoothness weight α .

OUT a dense flow field (u, v) at *every* pixel.

Flow (in words)

1. **Missing equation** $\rightarrow$ instead of a window, add a global prior: neighbouring pixels should move similarly.
2. Write one energy over the whole image: (brightness error) 2 + α ·(flow roughness) 2 .
3. Euler–Lagrange $\Rightarrow$ a huge but very sparse linear system (≈ 7 non-zeros per row).
4. Solve it iteratively: each pixel repeatedly pulls itself toward its neighbours’ average, corrected by the brightness residual.

Math

$$\min_{u, v} \iint \underbrace{(I_x u + I_y v + I_t)^2}_{\text{data term}} + \alpha \underbrace{(|\nabla u|^2 + |\nabla v|^2)}_{\text{smoothness prior}} dx dy$$

Iteration: $u \leftarrow \bar{u} - I_x \frac{I_x \bar{u} + I_y \bar{v} + I_t}{\alpha + I_x^2 + I_y^2}$, and the same with I_y for v .

where: the data term punishes flow that does not explain the observed brightness change; $|\nabla u|^2$ = how fast the flow field itself varies in space (large = torn/choppy field); α = the exchange rate between the two; $\bar{u}, \bar{v}$ = local averages of the current flow (this is where the neighbours enter); the denominator $\alpha + I_x^2 + I_y^2$ automatically trusts the data more where the gradient is strong.

Knobs

- $\alpha \uparrow$ — smoother, more complete field; washes out real motion boundaries and slows down genuine fast objects. Formally: more prior = MAP.
- $\alpha \downarrow$ — follows the data closely (ML), but noisy and full of holes in textureless regions.
- **iterations** $K \uparrow$ — closer to the exact solution; information spreads about 1 pixel per iteration, so large flat areas need many.

Exam trap the quadratic smoothness term *blurs motion across object boundaries*. Every “modify HS” exam question is fixed by making the smoothness weight per-edge: $\alpha w(p, q)$ with $w \approx 0$ across the given boundary signal (label change, depth jump, edge map) $\Rightarrow$ a **weighted graph Laplacian** instead of the plain one. A per-pixel *confidence* $U(x)$ instead multiplies the *data* term.

3 Ch.3 — Applications of Motion Estimation

Super-resolution (ML and MAP)

Ch.3

IN several low-res frames $\{Y_i\}$ of the same scene with *different sub-pixel* shifts; blur H , decimation D , motions F_i .

OUT one high-res image X .

Flow (in words)

1. Write the *forward* model — how a HR image becomes each LR frame: warp, blur, downsample, add noise.
2. Guess an X (e.g. upsampled first frame), push it forward, compare with each real LR frame.
3. Back-project the residual through the transposed operators and take a gradient-descent step.
4. Repeat. If it rings, add a smoothness prior (that is MAP instead of ML).

Math

$$Y_i = DHF_i X + v_i; \quad X_{n+1} = X_n - \beta \sum_i F_i^T H^T D^T (DHF_i X_n - Y_i) \quad [+ \gamma L(X_n) \text{ for MAP}]$$

where: F_i = motion of frame i (the sub-pixel shift that gives new information); H = camera blur (PSF); D = decimation, i.e. throwing away pixels; v_i = sensor noise; $F_i^T H^T D^T$ = the same three operations run backwards (smear the error back onto the HR grid); β = step size; L = Laplacian, the prior that punishes roughness; γ (or λ) = prior strength.

Knobs

- $\beta \uparrow$ — converges faster, can diverge/oscillate.
- $\lambda, \gamma \uparrow$ — smoother, kills the ringing of pure ML — but also kills real detail. $\downarrow$: sharper and noisier.
- **# frames** $\uparrow$ — you need roughly r^2 frames for an $r \times$ upscale, *and* their sub-pixel offsets must differ.

Exam trap identical sub-pixel offsets add *zero* information no matter how many frames. Camera shake actually helps.

RANSAC + BRIEF

Ch.3 • the plumbing of feature matching

IN a set of candidate matches that contains outliers (RANSAC); a keypoint window (BRIEF).

OUT a model fitted only to the inliers; a binary descriptor string.

Flow (in words)

(RANSAC)

1. Draw the *minimal* number of points n that defines the model (2 for a line, 3 for affine, 4 for homography).
2. Fit the model on just those, count how many other points agree within threshold t (inliers).
3. Repeat K times, keep the model with the most votes, then refit on *all* its inliers.

Flow (in words)

(BRIEF) fix a pattern of pixel pairs in the window; write bit 1 if the first pixel is brighter than the second, else 0; concatenate. Match by Hamming distance = XOR + popcount.

Math

$$K = \frac{\log(1 - P)}{\log(1 - W^n)}$$

where: W = fraction of matches that are inliers; n = sample size; P = probability you want of hitting at least one all-inlier sample; K = required number of draws. W^n = chance one draw is all-inliers — this is why n must be *minimal*.

Knobs

- t (**inlier threshold**) $\uparrow$ — more points counted as inliers, looser and possibly wrong fit. $\downarrow$: stricter but may find no consensus.
- $W \downarrow$ or $P \uparrow$ — K explodes — RANSAC dies at very high outlier ratios.
- **BRIEF # pairs** $\uparrow$ — more distinctive descriptor, slower matching and more memory.

Exam trap one bad match destroys a least-squares fit — always RANSAC *before* LS. BRIEF is fast and lighting-robust but **not rotation invariant**.

Hyperlapse (cost + dynamic programming)

Ch.3

IN a long shaky video; target speed-up v ; window w ; weights λ_s, λ_a .

OUT a *selected subset* of frames (not evenly spaced) that plays back smooth and fast.

Flow (in words)

1. Why not keep every k -th frame? Because the camera bobs, so uniform sampling lands at random phases of the bob $\Rightarrow$

jitter.

2. Define a cost for “how smoothly does frame j follow frame i ”: match Harris+BRIEF features, fit an affine transform with RANSAC, measure the leftover misalignment.
3. Add two penalties: missing the target speed-up, and changing pace abruptly.
4. Find the cheapest path through the frames with a 2-pass dynamic program (build cost table, back-trace).

Math

$$C(h, i, j, \mu) = C_m(i, j) + \underbrace{\lambda_s \min(\|(j - i) - \mu\|^2, T_s)}_{C_s: \text{speed}} + \underbrace{\lambda_a \min(\|(j - i) - (i - h)\|^2, T_a)}_{C_a: \text{acceleration}}$$

where: C_m = motion/alignment cost between frames i and j ; μ (or v) = desired frame gap; h, i, j = the previous, current and candidate next frame (three frames are needed because acceleration involves two gaps); T_s, T_a = truncations that stop one bad pair from dominating.

Knobs

- $\lambda_s \uparrow$ — playback speed sticks to target, at the price of jitter. In practice $\lambda_s \gg \lambda_a$: smoothness matters more than exact timing.
- $\lambda_a \uparrow$ — more uniform pacing (no sudden slow-motion patches).
- **window** $w \uparrow$ — DP may skip further ahead $\Rightarrow$ better paths, quadratically costlier.

Exam trap the output has *non-uniform* time steps. If the question demands exact playback timing, uniform sampling wins.

4 Ch.4 — Online Tracking: Kalman & Particle

Online = frames arrive one at a time, you may not store the video. Every filter is **predict** + **correct**.

Bayesian recursive tracking (the derivation)

Ch.4 • exam slot 2

IN measurements $z_1..z_t$; a motion model; a measurement model.

OUT the posterior belief $p(x_t | z_{1:t})$ over the current state, computed *recursively* (no stored history).

Flow (in words)

1. Assume **Markov**: the future depends only on the present, $p(x_t | x_{1:t-1}) = p(x_t | x_{t-1})$.
2. Assume **conditional independence** of measurements: z_τ depends only on x_τ .
3. Apply Bayes — the two assumptions collapse the whole history into three factors.
4. Marginalise out the past to get the filtering form. Kalman and particle filters are just two ways to *represent* this same distribution.

Math

$$p(x_{1:t} | z_{1:t}) \propto \underbrace{p(z_t | x_t)}_{\text{likelihood}} \underbrace{p(x_t | x_{t-1})}_{\text{motion model}} \underbrace{p(x_{1:t-1} | z_{1:t-1})}_{\text{previous posterior = recursion}}$$

$$p(x_t | z_{1:t}) = C p(z_t | x_t) \int p(x_t | x_{t-1}) p(x_{t-1} | z_{1:t-1}) dx_{t-1}$$

where: x_t = state (position, velocity, size...); z_t = this frame's noisy measurement; the likelihood scores how well the state explains the image (e.g. $e^{-\text{SSD}}$ of a template); the integral is the *prediction* step (push the old belief through the motion model); C = normaliser.

Exam trap if asked to “derive the recursion”, the whole answer is: state the two assumptions, apply Bayes, name the three factors. That is the marking scheme.

Kalman filter

Ch.4

IN Φ, H, Q, R ; initial $\hat{x}_0, P_0$; a measurement z_k each frame.

OUT filtered state $\hat{x}_k$ and its uncertainty P_k , per frame.

Flow (in words)

1. **Predict**: push the state through the motion model. Uncertainty *grows* by Q (the model is not perfect).
2. **Gain**: compare predicted uncertainty against measurement noise $R \Rightarrow$ how much to trust the new data.
3. **Measure**: get z_k (LK flow, template match near the prediction).
4. **Update**: move the estimate toward the measurement by gain $\times$ (innovation). Uncertainty *shrinks*.

Math

$$\begin{aligned} \hat{x}'_k &= \Phi \hat{x}_{k-1}, & P'_k &= \Phi P_{k-1} \Phi^T + Q \\ K_k &= P'_k H^T (H P'_k H^T + R)^{-1} \\ \hat{x}_k &= \hat{x}'_k + K_k (z_k - H \hat{x}'_k), & P_k &= (I - K_k H) P'_k \end{aligned}$$

where: $\hat{x}'_k$ = prediction *before* seeing data; $\hat{x}_k$ = estimate *after*; P = error covariance (small = confident); Φ = state transition, e.g. $[x, v] \mapsto [x + v\Delta t, v]$; H = measurement matrix (which part of the state you actually observe); Q = how much the real world deviates from Φ ; R = how noisy the sensor is; $z_k - H \hat{x}'_k$ = *innovation*, the surprise; K_k = the trust dial. 1-D sanity check: $K = P'/(P' + R)$, so $R \rightarrow 0 \Rightarrow K \rightarrow 1$ (follow the data), $R \rightarrow \infty \Rightarrow K \rightarrow 0$ (ignore the data).

Knobs

- $R \uparrow$ — trust the model, output very smooth but laggy; $\downarrow$: snaps to every noisy measurement.
- $Q \uparrow$ — trust the model less, filter reacts fast to manoeuvres but is jumpy; $\downarrow$ too far: over-confident, drifts away and never recovers after occlusion.
- **state vector** — adding velocity/acceleration lets the tracker *coast* through an occlusion instead of freezing.

Exam trap Kalman is optimal **only** for linear dynamics + Gaussian noise, and its belief is a single Gaussian — it cannot represent “the target is either here or there”. That is the particle filter’s job.

Particle filter (Condensation)

Ch.4 • exam slot 2

IN previous particle set $\{S_{t-1}^{(n)}, \pi_{t-1}^{(n)}, C_{t-1}^{(n)}\}_{n=1}^N$; new frame.

OUT new weighted particle set + a state estimate.

Flow (in words)

1. **Select (resample)**: draw $r \sim U[0, 1]$, pick the smallest j with $C^{(j)} > r$. Heavy particles get duplicated, light ones die. *This is inverse-CDF sampling — the loaded-die question.*
2. **Predict (drift + diffuse)**: $S_t = AS'_t + Bw_t$. Deterministic drift moves the cloud; random diffusion spreads it so new hypotheses appear.
3. **Measure**: weight each particle by how well the image supports it, $\pi = p(z_t | x_t)$, e.g. $e^{-\text{SSD}}$ against the template.
4. **Normalise** weights to sum to 1 and rebuild the cumulative array C . Output the weighted mean (or the max-weight particle).

Math

$$\pi_t^{(n)} = p(z_t | x_t = S_t^{(n)}), \quad C_t^{(n)} = C_t^{(n-1)} + \pi_t^{(n)}, \quad \hat{x}_t = \sum_n \pi_t^{(n)} S_t^{(n)}$$

where: $S^{(n)}$ = the n -th hypothesis (a full state); $\pi^{(n)}$ = its weight = how believable it is; $C^{(n)}$ = running cumulative sum, i.e. a discrete CDF, which is what makes weighted sampling a single-comparison operation; A = deterministic motion; Bw_t = injected noise.

Knobs

- N (**particles**) $\uparrow$ — better approximation of any-shaped belief, linear cost increase; $\downarrow$: *sample impoverishment* — the cloud collapses onto one wrong mode.
- **diffusion noise** $\uparrow$ — explores more, survives sudden manoeuvres; too much: the estimate rattles and the weights spread thin.
- **output rule** — max-weight = sharp but jittery; weighted mean = stable but can land *between* two modes, i.e. on nothing.

Exam trap “sample from a distribution using only **rand()**” = build the CDF, draw one uniform, invert. Same answer for a loaded die, a discrete pmf, or a continuous F ($x = F^{-1}(r)$).

5 Ch.5 — Dynamic Texture Synthesis

IN a short video of a *stochastic, stationary* texture (fire, smoke, water, grass); state dimension n .

OUT an endless stream of new, never-repeating frames with the same statistics.

Flow (in words)

1. **Vectorise**: each frame becomes one long column vector; stack them into a matrix Y .
2. **Find the low-dimensional space**: SVD of Y , keep the top n components ($\geq 95\%$ energy). U = the spatial patterns, ΣV^T = how strongly each pattern is on over time = the hidden state trajectory.
3. **Learn the dynamics**: least-squares fit of A such that today's state predicts tomorrow's.
4. **Synthesise**: start from some state, repeatedly apply A *plus noise*, and render each state back to an image with C . Without noise the trajectory decays or loops.

Math

$$x(t+1) = Ax(t) + Bv(t), \quad y(t) = Cx(t) + w(t); \quad Y = U\Sigma V^T \Rightarrow \hat{C} = U, \quad \hat{X} = \Sigma V^T, \quad \hat{A} = \arg \min_A \|X_1^T - AX_0^{T-1}\|$$

where: $y(t)$ = the actual frame ($m \approx 200 \times 300$ numbers); $x(t)$ = the hidden state ($n \approx 50$ numbers) = a compressed description of “what the fire is doing right now”; C = decoder from state to image; A = the physics: how the state evolves; v, w = state noise and image noise; the constraint $C^T C = I$ removes the ambiguity $A \rightarrow TAT^{-1}$.

Knobs

- n (state dim) $\uparrow$ — sharper, more faithful texture, more computation and risk of memorising the input; $\downarrow$: blurry mush.
- state noise v $\uparrow$ — more variety, never repeats; too much: unnatural, unstable trajectory. Zero noise $\Rightarrow$ deterministic and it dies out.
- image noise w — hides the low-rank “plastic” look; too much: grainy.

Exam trap works only for *statistically stationary* content. Feed it a walking person and you get blurry blobs — the linear low-rank model has no notion of structure.

Domain-transfer version (the actual exam question): text with Word2Vec (or a melody with note embeddings). Word $\rightarrow$ vector replaces frame $\rightarrow$ SVD-vector; fit A on the embedding sequence; run forward with noise; decode by nearest-neighbour in the vocabulary. Noise is essential; the result copies local style, not grammar.

6 Ch.6 — Layer Representation

IN a video (or frame pair) built from a few coherently moving layers; the number of layers k .

OUT per-pixel layer assignment + one affine motion per layer + the layer images and masks.

Flow (in words)

1. Synthesis model: a movie is a stack of layers glued by alpha compositing, $I_k = I_{k-1}(1 - \alpha_k) + E_k\alpha_k$.
 2. To invert it, assume each layer moves as *one affine transform* = a single point in $\mathbb{R}^6$.
 3. Fit LK affine motion on many random windows $\Rightarrow$ a cloud of points in $\mathbb{R}^6$.
 4. K-means that cloud into k groups = the layers. Refit LK per layer using only its pixels. Repeat.
- This is the classic *assume A to estimate B, then swap* (EM-style) alternation.

Math

$$I = \alpha E + (1 - \alpha)I_{\text{prev}}, \quad V \in \mathbb{R}^6 \text{ per window}$$

where: E = the layer's own image (colour); $\alpha \in [0, 1]$ = its mask/opacity (1 = fully covers, 0 = invisible); V = the 6 affine parameters, used here as a *feature vector* to cluster, not as a warp.

Knobs

- k $\uparrow$ — one real object gets split into several layers; $\downarrow$: two differently-moving objects get merged. MDL (minimum description length) is the principled way to pick k .
- **motion model** — affine (6) is usually enough; homography (8) if a layer is a plane under strong perspective.
- **# random windows** $\uparrow$ — denser sampling of motions, slower.

Exam trap the clustering happens in *motion space* $\mathbb{R}^6$, not colour space.

IN points $\{x_i\} \subset \mathbb{R}^d$; number of clusters k ; initial centres.

OUT k centres + an assignment of every point.

Flow (in words)

1. Assign: every point goes to its nearest centre.
2. Update: every centre moves to the mean of its members.
3. Repeat until assignments stop changing.

Math

$$J = \sum_i \|x_i - c_{\text{enc}(i)}\|^2, \quad \frac{\partial J}{\partial c_j} = 0 \Rightarrow c_j = \frac{1}{|S_j|} \sum_{i \in S_j} x_i$$

where: J = distortion, the total squared error; $\text{enc}(i)$ = index of the centre point i belongs to; the derivative shows *why* the update step is the mean — it is the exact minimiser for fixed assignments.

Termination proof (write this exactly): (a) The assignment step cannot increase J — each point moves to a centre at most as far as its current one. (b) The update step cannot increase J — the mean is the minimiser. So J is monotonically non-increasing. (c) There are at most k^n possible assignments, each with a fixed J . Since J never increases, a changed assignment must have strictly lower J (use a fixed tie-break, e.g. lowest index, for equidistant points), so no assignment can repeat. A monotone sequence over finitely many values must stop $\Rightarrow$ terminates in finite time. Caveat: only a *local* minimum; global k -means is NP-hard.

Knobs

- $k \uparrow$ — distortion always drops (it hits 0 at $k = n$) — so you cannot pick k by minimising J ; use MDL.
- **initialisation** — decides which local minimum; k -means++ avoids empty clusters.
- **restarts** $\uparrow$ — keep the lowest-distortion run.

Exam trap “Can it cycle forever?” No — monotone + finite. “Can J increase?” No. “Ties?” Need a deterministic tie-break, otherwise the strictness argument breaks.

7 Ch.7 — Video Magnification

Eulerian video magnification

Ch.7

IN a video from a *steady* camera; amplification factor α ; a temporal frequency band.

OUT the same video with tiny motions amplified (pulse in a face, a swaying building).

Flow (in words)

1. *Lagrangian* would mean tracking each pixel (motion estimation — hard and fragile). **Eulerian** instead sits still at each fixed location and only watches its intensity over time.
2. Band-pass that per-pixel temporal signal to the frequency you care about (≈ 1 Hz for a heart beat). Everything else (lighting drift, noise) is discarded.
3. Multiply the band-passed part by α and add it back to the original.
4. Because intensity change $\approx$ displacement $\times$ spatial slope, boosting the change is the same as boosting the displacement — *without ever estimating motion*.

Math

$$I(x, t) = f(x + \delta(t)) \approx f(x) + \underbrace{\delta(t) \frac{\partial f}{\partial x}}_{B(x, t)} \Rightarrow \tilde{I} = I + \alpha B \approx f(x + (1 + \alpha)\delta(t))$$

where: $\delta(t)$ = the true (sub-pixel) displacement you cannot see; $\partial f / \partial x$ = the spatial gradient that converts displacement into a brightness change; B = the temporal variation actually measured, in practice $\approx I_2 - I_1$ after band-passing; α = gain.

Knobs

- $\alpha \uparrow$ (**typ. 5–50**) — reveals smaller motions but amplifies noise and produces ringing/halos once the Taylor approximation breaks.
- **band-pass band** — narrow around the target frequency = clean; too wide = you amplify noise and illumination drift.
- **pyramid** — amplify each spatial scale separately so that motion larger than ~ 1 pixel does not ring.

Exam trap the first-order Taylor step assumes motion below ~ 1 pixel. Big motion, or a shaky camera, gets amplified into garbage — the camera must be on a tripod.

8 Ch.8 — Background Subtraction

Per-pixel Gaussian Mixture Model (GMM)

Ch.8

IN a video stream from a *static* camera; K Gaussians per pixel; rates α, ρ ; background threshold T .

OUT a binary foreground/background label per pixel, per frame, computed online.

Flow (in words)

1. *Motivation*: look at one pixel's value across all frames (its **intensity profile**). It is not one constant — a leaf blowing over sky makes it flip between several typical values. So model each pixel as a *mixture* of a few Gaussians.
2. Batch clustering of a stored history would work but costs delay, memory and compute. Instead update *incrementally*: no history stored.
3. For each new value: if it lies within 2.5σ of an existing Gaussian, nudge that Gaussian's mean and variance toward it and raise its weight. Otherwise replace the weakest Gaussian with a new one centred on this value (low weight, high variance).
4. Sort Gaussians by ω_k / σ_k (frequent *and* tight = background). Take the top ones until their weight sum passes T : those are background. Anything else = foreground.

Math

$$f(x) = \sum_k \omega_k \mathcal{N}(x; \mu_k, \sigma_k), \quad \omega_{k,t} = (1 - \alpha)\omega_{k,t-1} + \alpha M_{k,t}, \quad \mu_t = (1 - \rho)\mu_{t-1} + \rho x_t, \quad \sigma_t^2 = (1 - \rho)\sigma_{t-1}^2 + \rho(x_t - \mu_t)^2$$

$$B = \arg \min_b \left(\sum_{k \leq b} \omega_k / \sigma_k > T \right)$$

where: ω_k = how often this mode is seen (popularity); μ_k, σ_k = its colour and its spread; $M_{k,t} \in \{0, 1\}$ = did the new value match Gaussian k ; α = learning rate for the weights (how fast a new pattern earns background status); ρ = learning rate for mean/variance; T = what fraction of the observations must be explained before you stop calling modes “background”. The ratio ω/σ is the key ranking: background is both *common* and *stable*.

Knobs

- K (3–5) $\uparrow$ — models more background modes (leaves + sky + shadow), but starts fitting noise and slows down.
- $\alpha \approx 0.01$ $\uparrow$ — adapts quickly to lighting change, but a car that parks becomes background in seconds (and a slow-moving object is absorbed); $\downarrow$: keeps ghosts of things that already left.
- T (0.7–0.9) $\uparrow$ — more modes counted as background $\Rightarrow$ fewer false alarms, more missed detections.
- **match radius** 2.5σ $\uparrow$ — noise gets absorbed instead of flagged; $\downarrow$: hyper-sensitive, everything is foreground.

Exam trap fails on sudden global illumination change, camera shake, and shadows (a shadow is a genuine colour change, so it is flagged as foreground).

9 Ch.9 — Colorization & Geodesic Distance

The whole chapter is one idea: **redefine distance so that it follows the image content**. Moving inside a homogeneous region is nearly free; crossing an edge is expensive.

Geodesic distance colorization / label propagation

Ch.9 • exam slots 6 & 7

IN a grey (luminance) image or video Y ; a few user scribbles $\{\Omega_k\}$ each with a colour/label c_k ; exponent b .

OUT a colour (or a soft label mixture) for *every* pixel.

Flow (in words)

1. Build a graph: pixels are nodes, neighbours are edges, edge weight = $|Y(p) - Y(q)|$ (how much brightness changes when you step there).
2. For each scribble k , run Dijkstra from that whole scribble set at once $\Rightarrow$ a distance map $D_k(x)$ over the image.
3. Every pixel votes: nearby scribbles get big weight, far ones almost none.
4. Normalise the votes so they sum to 1 $\Rightarrow$ a convex blend of the scribble colours.

Math

$$L(p) = \int_p |\langle \dot{p}(s), \nabla Y \rangle| dl, \quad D_k(x) = d_L(x, \Omega_k), \quad c(x) = \frac{\sum_k W(D_k(x)) c_k}{\sum_k W(D_k(x))}, \quad W(D) = \frac{1}{D^b}$$

where: $\langle \dot{p}, \nabla Y \rangle$ = the component of the image gradient *along* the direction you are walking — so walking parallel to an edge is cheap, crossing it is expensive; $D_k(x)$ = geodesic distance from pixel x to scribble k ; $W(D) = D^{-b}$ = the vote weight; the denominator makes the weights sum to 1, which is exactly why the output is a valid colour (or a valid percentage mix).

Knobs

- $b \in [1, 6]$ $\uparrow$ — votes concentrate on the single nearest scribble $\Rightarrow$ sharp, almost hard boundaries; $\downarrow$: soft, washed-out blends between scribbles.
- **edge weight definition** — $w = |\nabla Y|$ makes edges into *walls* (colour cannot leak). Invert it and paths *ride* the edges instead — see the trap.
- **scribble placement** — put them *inside* regions, never on an edge; one per distinct region.
- **neighbourhood** — 4-neighbours in an image; **6-neighbours in video** (adds $\pm t$) so colour flows through time — this is why video needs far fewer scribbles than a stack of separate images.

Exam trap The sign of the edge weight is the whole question.

- Want the path/colour to *avoid* edges (colorization, “stay in the middle of the road”): $w = |\nabla I|$, crossing is expensive.
- Want the path to *follow* an edge (“intelligent scissors”, trace Lena’s hat brim): $w = 1/(|\nabla I| + \varepsilon)$ or $\max |\nabla I| - |\nabla I|$, so riding a strong contour is nearly free. Same Dijkstra, inverted cost.

Domain-transfer versions (exam slot 7): 3D point cloud $\Rightarrow$ build a k -NN graph, weight = $\|p - q\|$ plus normal difference, then identical Dijkstra + $\arg \min_k D_k$. Social graph with tie strengths $w(u, v) \Rightarrow$ edge *length* $\ell = 1/w$ (or $-\log w$), then the same $W(D_k)/\sum_j W(D_j)$ blend gives “30% finance / 70% fashion”.

Dijkstra vs Fast sweeping (Danielsson)

Ch.9

IN a weighted grid/graph; a source set Ω_s .

OUT $D(x)$ = geodesic distance from the source set to every node.

Flow (in words)

(Dijkstra)

1. $D = 0$ on the sources, ∞ elsewhere; all nodes unvisited.
2. Pop the unvisited node with the smallest D (priority queue).

3. Relax its neighbours: $D(y) \leftarrow \min\{D(y), D(x) + w(x, y)\}$. Remove x . Repeat.

Mental picture: water spreading from the sources, flowing slowly through high-gradient areas; the moment a pixel gets wet is its distance. Cost $O(N \log N)$. Correct because of the **Bellman principle**: every sub-path of an optimal path is optimal.

Flow (in words)

(Fast sweeping) forget the queue: run plain raster scans over the image, updating each pixel from its already-visited neighbours. Four sweeps cover the four diagonal directions; repeat until nothing changes.

Knobs

- **Dijkstra** — exact, single-source, CPU-friendly. The priority queue is inherently sequential $\Rightarrow$ hard to parallelise.
- **Fast sweeping # sweeps** $\uparrow$ — handles more path turns; 4 sweeps suffice for simple paths, a spiral with N turns needs $\geq 4N$. Cache- and GPU-friendly, usually faster in practice.
- **neighbourhood size** $\uparrow$ — more possible path directions (less grid bias), higher cost per node.

Exam trap “design it for a GPU” $\Rightarrow$ fast sweeping, not Dijkstra. “I need the exact single-source map on a CPU” $\Rightarrow$ Dijkstra.

10 Ch.10 — Video Matting

Goal: cut the foreground out and paste it on a new background *without* dragging the old background’s colour along. Three stages, all built on Ch.9 geodesics.

Stage 1 — Trimap estimation (KDE + geodesic)

Ch.10

IN image/video; FG and BG scribbles; band radius ρ ; KDE bandwidth σ .

OUT a trimap: 1 = definitely foreground, 0 = definitely background, ϕ = unsure.

Flow (in words)

1. From the scribbled pixels build colour distributions for FG and BG using KDE (drop a Gaussian bump on every scribbled colour and sum).
2. Bayes turns the two densities into $P(F | c)$, the probability that colour c is foreground.
3. Compute geodesic distances D_F, D_B using the *gradient of that probability map* as the cost field (so the walls sit where foreground-ness changes, not where brightness changes).
4. Label x foreground if $D_F(x) < D_B(x)$. Mark a band of radius ρ around the tie surface $\{D_F = D_B\}$ as “unsure”.

Math

$$f(c | F) \propto \sum_{x \in \Omega_F} K(c, c(x)), \quad K \propto e^{-\|c - c'\|^2 / 2\sigma^2}; \quad P(F | c) = \frac{f(c | F)}{f(c | F) + f(c | B)}; \quad W(x) = \nabla_x P(F | c(x))$$

where: K = the Parzen kernel, a soft bell so that a colour close to a scribbled colour still counts; σ = its width; the ratio is Bayes with a uniform prior; $W(x)$ = the cost field for Dijkstra — large where foreground-probability changes fast, i.e. at the true object boundary.

Knobs

- ρ (**band radius, 5–15 px**) $\uparrow$ — captures wide fuzzy edges like hair, but leaves far more work (and more error) to stage 2; $\downarrow$: fast, but crops hair off.
- σ (**KDE width**) $\uparrow$ — smoother colour model, generalises to unseen shades; too large: FG and BG distributions merge into mush.
- **scribbles** — one per *distinct colour region* of each side, otherwise that region has no density support.

Exam trap colour alone is ambiguous when FG and BG share colours — the *geodesic* term is what breaks the tie (spatial reachability), which is the point of the whole design.

Stage 2 — Opacity map & Stage 3 — Matting

Ch.10

IN the trimap, the image, a new background B .

OUT a continuous $\alpha(x) \in [0, 1]$ inside the band, and the final composite J with no colour bleed.

Flow (in words)

1. **Opacity**: inside the narrow band there are far more samples than the sparse scribbles, so *re-estimate* FG/BG colour models *locally* along the boundary curve. Combine local probability with geodesic distance $\Rightarrow$ a soft α .
2. **Matting**: a fuzzy pixel is physically a mix, $c = \alpha f + (1 - \alpha)b$: one equation, two unknown colours. Search a small window for the pair (confident-FG pixel, confident-BG pixel) that best reproduces the observed mixed colour.
3. Composite the recovered *pure* foreground colour onto the new background.

Math

$$\alpha(x) = \frac{w_F(x)}{w_F(x) + w_B(x)}, \quad w_F = \tilde{D}_F^{-r} \tilde{P}_F, \quad r \in (0, 2]; \quad (x_F^*, x_B^*) = \arg \min \|\alpha c(x_F) + (1 - \alpha)c(x_B) - c(x)\|^2$$

$$J(x) = \alpha c(x_F^*) + (1 - \alpha)B(x)$$

where: $\tilde{D}_F$ = local geodesic distance to confident foreground; $\tilde{P}_F$ = locally re-estimated foreground probability; r = how strongly proximity beats colour evidence; $c(x_F^*)$ = the recovered *pure* foreground colour; $B(x)$ = the new background.

Knobs

- $r \uparrow$ — α is decided mostly by which side is geometrically closer $\Rightarrow$ crisper matte; $\downarrow$: colour evidence dominates, softer and noisier.

- **window size for the (f, b) search** $\uparrow$ — more candidate pairs $\Rightarrow$ better explanation of the mixed colour, quadratically slower.

Exam trap the naive composite $J = \alpha c(x) + (1 - \alpha)B$ is *wrong*: $c(x)$ already contains the old background, so its colour bleeds onto the new one (green screen halo). You must recover the pure f first. This is the point of stage 3.

Fast KDE via a low-rank kernel

Ch.10 • efficiency slot

IN a kernel $k(x, y)$ and weights $w(y)$; want $f(x) = \sum_y k(x, y)w(y)$ for all x .

OUT the same values in $O((m + n)p)$ instead of $O(mn)$.

Flow (in words)

factor the kernel matrix as a sum of p rank-1 terms $K = \sum_{i=1}^p \phi_i \psi_i^T$. Then the double sum splits into two cheap passes: first collapse the weights, $a_i = \sum_y \psi_i(y)w(y)$ (one pass over y), then expand, $f(x) = \sum_i a_i \phi_i(x)$ (one pass over x).

where: p = effective rank; ϕ_i, ψ_i = the separable factors. Gaussians are formally infinite rank but their singular values decay fast, so a small p is enough (this is the Fast Multipole idea).

Knobs

$p \uparrow$: more accurate, slower. $\downarrow$: faster, coarser.

Exam trap this is the generic answer to any “make it efficient” sub-question involving all-pairs sums.

11 Ch.11 — Retargeting & PatchMatch

Bidirectional similarity retargeting

Ch.11

IN source image/video S ; a target size; balance α ; patch size.

OUT target T of the new size that looks natural.

Flow (in words)

1. Two demands, in opposite directions. **Completeness**: every patch of S should appear somewhere in T (nothing important is lost). **Coherence**: every patch of T should come from S (nothing is invented / no artifacts).
2. Sum them with weight α ; this is the objective.
3. Initialise T by a *gradual* pyramid resize (never from black — you would land in a garbage local minimum).
4. Iterate: for each target patch find its NN in the source and vote its colour in; for each source patch find its NN in the target and vote in too; divide by the accumulated weight. That average is the exact per-pixel minimiser.

Math

$$D_{\text{com}}(S, T) = \frac{1}{N_S} \sum_{P \in S} \min_{Q \in T} d^2(P, Q), \quad D_{\text{coh}}(S, T) = \frac{1}{N_T} \sum_{Q \in T} \min_{P \in S} d^2(P, Q)$$

$$D = \alpha D_{\text{com}} + (1 - \alpha) D_{\text{coh}}$$

where: P, Q = patches (5–7 px, or $5 \times 5 \times 3$ space-time cubes for video); $d^2 = \|P - Q\|_2^2$; N_S, N_T = patch counts, which normalise for the size difference; each min is a nearest-neighbour query — that is the bottleneck PatchMatch solves.

Knobs

- $\alpha \uparrow$ — more content is forced to survive (nothing gets cropped), at the cost of visible squeezing/artifacts; $\downarrow$: cleaner-looking result that may silently delete objects. Start at 0.5.
- **patch size** $\uparrow$ — more context, more structural consistency, blurrier output; $\downarrow$: sharper but incoherent.
- **constraint weight** W_P — high on a face or a straight line = protect it; ≈ 0 on an object = it cannot vote, so it vanishes and neighbours fill its place. Weights keep D convex.
- **pyramid step** ($\sim 1.1 \times$) — gradual = good initialisation; aggressive = bad local minima.

Exam trap the directed distance is **not symmetric**: with $P = \{0\}$, $Q = \{0, 10\}$, $\max_P \min_Q d = 0$ but the reverse is 10. That asymmetry is exactly *why* both completeness and coherence are needed.

PatchMatch

Ch.11 • exam slot 5

IN source S , target T ; patch size; search window W ; iterations K (4–6).

OUT an approximate nearest-neighbour *offset field* $f(x)$: for every patch, where its best match lives.

Flow (in words)

1. **Init**: give every patch a random offset. Almost all are terrible — but with millions of patches, *some* are accidentally good.
2. **Propagation**: if patch (100, 100) matches at (200, 50), its right-hand neighbour almost certainly matches at the same offset. So each patch tests its own offset plus its left and upper neighbours' offsets, and keeps the best. Good matches spread across the image like a flood.
3. **Random search**: test a few random offsets around the current one, with an *exponentially shrinking* radius. This escapes local minima and refines.
4. Alternate the raster scan direction each pass (so information can also travel up/left). Converges in 4–6 passes, orders of magnitude faster than brute force.

Math

$$f(x) = \arg \min_y \|P_x - Q_{x+f}\|_2^2; \quad f(x) \leftarrow \arg \min_{f \in \{f(x), f(x-e_1), f(x-e_2)\}} \|P_x - Q_{x+f}\|$$

where: $f(x)$ = the offset (a displacement vector, not an absolute position — that is what makes propagation meaningful);

e_1, e_2 = one step left / one step up (the already-processed causal neighbours); the shrinking search radius starts at the image size and halves each step.

Knobs

- **iterations** $K \uparrow$ — better NN field, linearly slower; most of the gain is in the first 2 passes.
- **patch size** $\uparrow$ — more context and fewer false matches, blurrier synthesis.
- **random-search radius** $\uparrow$ — escapes bad local minima more often, slower convergence.

Exam trap PatchMatch relies on *spatial coherence* of matches. If correspondences are genuinely unstructured, propagation buys nothing. It is approximate, never exact NN.

The three classic extensions (know all three):

- **Clone detection**: run PatchMatch of the image *against itself*, forbidding the trivial near- x match. A cloned region shows up as a large connected component with $d \approx 0$ and one shared dominant offset v (its twin has $-v$). In video: 3D self-PatchMatch, and a duplicated segment appears as a constant *temporal* offset.
- **k -NN**: keep a size- k max-heap of offsets per patch instead of one. Propagation pools *all* k offsets of x , of the left and of the upper neighbour and keeps the k best. Random search inserts and evicts the worst. Critical subtlety: enforce **distinctness**, else the heap fills with k near-copies of one match. Cost $\approx k \times$.
- **Multi-image**: let the offset carry an image index (m, y) ; propagation borrows the neighbour's (m, y) , random search perturbs y and occasionally re-rolls m .

Inpainting, reshuffling, object removal

Ch.11

IN the same objective, plus a mask saying which pixels are *fixed* and which are *free*, plus patch weights.

OUT the edited image.

Flow (in words)

all three are one algorithm: decide what is frozen, minimise completeness + coherence over the rest with PatchMatch.

- **Inpainting**: free = the hole Ω_m ; frozen = everything else. Make the mask a few pixels *generous* to avoid halos from contaminated edge pixels.
- **Reshuffling**: freeze the pasted object at its new place, let the algorithm re-synthesise everything around it.
- **Removal**: set the object's patch weights ≈ 0 so it cannot vote anywhere $\Rightarrow$ it evaporates and neighbours fill in. **Preservation**: set weights high on the patches you must keep.

Exam trap patch methods copy, they cannot hallucinate. A large hole over unique structure (a face, text) cannot be filled correctly.

12 Ch.12 — Intrinsic Images (Weiss)

Weiss intrinsic images — median of derivative filters

Ch.12 • exam slot 3

IN T images of a *fixed* scene from a *fixed* camera, with only the lighting changing (a day of webcam frames); derivative filters $\{f_n\}$ (usually just ∂_x, ∂_y).

OUT one reflectance image $\hat{r}$ (the “paint”, constant over time) and T illumination images $\hat{l}(\cdot, t)$ (the “light”).

Flow (in words)

1. Take logs: multiplication becomes addition, $i = r + l$. Now the problem is linear.
2. Filter every frame with derivative filters. Key assumption: **illumination edges are sparse** — shadow boundaries are rare, so most filtered-lighting values are ≈ 0 .
3. At each pixel and each filter, take the **median over time** of the filtered frames. The reflectance gradient is present in *every* frame; the shadow edges are rare outliers, and the median throws outliers away.
4. You now have the *gradients* of the reflectance. Reconstruct the image itself by a fixed deconvolution (pseudo-inverse) filter g , then get each frame's lighting by subtraction.

Math

$$\hat{r}_n(x, y) = \text{median}_t o_n(x, y, t), \quad o_n = i \star f_n; \quad \hat{r} = g \star \left(\sum_n f_n^r \star \hat{r}_n \right) \text{ with } g \star \left(\sum_n f_n^r \star f_n \right) = \delta; \quad \hat{l}(\cdot, t) = i(\cdot, t) - \hat{r}$$

where: $i = \log I$; $r = \log$ reflectance (surface paint); $l = \log$ illumination; $f_n^r(x, y) = f_n(-x, -y)$ = the flipped filter (correlation instead of convolution); g = the deconvolution filter that turns filter responses back into an image — it depends *only* on the chosen filters, so precompute it once and reuse it forever; δ = the identity (delta) kernel.

Why the median (derivation in one line): assume the filtered lighting follows a Laplacian prior $P(x) \propto e^{-\alpha|x|}$ (sparse). Maximising that likelihood over t means minimising $\sum_t |o_n(t) - r_n|$, i.e. the ℓ_1 error — and the ℓ_1 minimiser of a set of numbers is the **median**. (An ℓ_2 /Gaussian prior would have given the mean, which shadows would corrupt.)

Convergence (Claim 2): with $p_\epsilon = P(|f_n \star l| < \epsilon)$, the probability of a good estimate is $\geq \sum_{k=0}^{\lfloor T/2 \rfloor} \binom{T}{k} (1 - p_\epsilon)^{T-k} p_\epsilon^k$. Sparser lighting = larger p_ϵ = faster convergence: $p_\epsilon = 0.85$ already gives ≥ 0.93 at $T = 3$ frames and ≥ 0.97 at $T = 5$.

Knobs

- **# frames** $T \uparrow$ — higher confidence (the binomial bound); 3–5 frames already work if lighting is sparse.
- **filters** $\{f_n\}$ — two derivative filters are enough; adding more improves conditioning but g must be recomputed for the new set.
- **sparseness of the lighting** — the single assumption everything rests on. Smooth, non-sparse illumination breaks it.

Exam trap take the median of **gradients**, never of raw pixels. A pixel permanently in shadow has no shadow *edge*, so its gradient median is clean — but a raw-pixel median (or min, or mean) would bake the shadow permanently into the “paint”.

Online version (the actual exam question, Moed A Q3): the only non-online part is the median. Keep, per pixel and per filter, a **256-bin histogram** of the quantised filter response. Each frame: compute the responses, increment one bin per pixel ($O(1)$), and read the running median as the smallest bin whose cumulative count reaches $t/2$. Then reconstruct with the precomputed g and emit R_t every frame. Memory = $256 \times (\#filters)$ per pixel — *independent of t* . The same trick with $\arg \max$ instead of the median gives the running *mode* (background modelling).

13 Slot 4 — The “paper” questions (outside the notes)

Evolving time fronts (Iguazu) & Light fields / concentric mosaics

slot 4

These two appeared as slot-4 questions and are *not* in the book. Both are answerable from Ch.1 and space-time reasoning; say so explicitly in the exam and then solve it with what you have.

Reverse the pan but keep the water falling down.

1. Two motions are entangled: the global camera pan (in x) and the local water motion (in t). Plain reverse playback reverses *both*, so the water flows up. They must be decoupled.
2. Stack all frames into an xyt volume. A normal output frame is a flat slice at $t = \text{const}$ — a “time front”.
3. Sweep a **slanted / non-planar time-front surface** through the volume: advance the surface in $-x$ (reversed pan) while the local time coordinate along it still *increases* (water keeps falling).
4. Each output frame is one cut of the volume with that surface.

Novel view from a camera on a rotating arm.

1. A rotating camera densely samples the scene's *rays*. A novel view is just a new bundle of rays through a virtual centre.
2. Calibrate: each frame k has a known projection matrix P_k (Ch.1 intrinsics + the known arm angle gives R_k, t_k). Define the virtual P_v .
3. For each virtual pixel, back-project to a 3D ray; find the recorded camera/pixel whose ray best coincides with it; copy that colour.
4. If the virtual ray falls between recorded rays, blend the nearest ones by angular closeness. This is image-based rendering / the light-field idea.

Exam trap for slot 4, points come from *reasoning with the course tools*, not from naming the paper. Start by identifying which two things are entangled, then say how you decouple them.

14 The derivations they actually ask for

Six short derivations — know the first line of each

1. **Optical flow constraint.** $I(x+u, y+v, t+1) = I(x, y, t)$; first-order Taylor $\Rightarrow I_x u + I_y v + I_t = 0$. One equation, two unknowns — *everything* in Ch.2 is a way to supply the second.
2. **LK normal equations.** Stack the constraint over a window: $Ax = b$, $A = [I_x \ I_y]$ rows, $b = -I_t$. Minimise $\|Ax - b\|^2$, set the derivative to zero $\Rightarrow A^T Ax = A^T b$. Solvable iff $A^T A$ (the structure tensor) has rank 2 $\Rightarrow$ corners $\Rightarrow$ Harris.
3. **Horn-Schunck.** Energy = data + $\alpha \cdot$ smoothness. Euler-Lagrange on the integral gives $(I_x u + I_y v + I_t)I_x - \alpha \nabla^2 u = 0$ (and the v twin). Discretising ∇^2 with the Laplacian kernel yields a sparse $2N \times 2N$ system, solved by the fixed-point iteration on the card. *To modify it:* replace ∇^2 by a **weighted graph Laplacian** where each neighbour contributes $-w(p, q)$.
4. **Bayes filtering recursion.** Markov + conditional independence + Bayes $\Rightarrow$ posterior $\propto$ likelihood $\times$ motion model $\times$ previous posterior. Marginalise the past $\Rightarrow p(x_t | z_{1:t}) = C p(z_t | x_t) \int p(x_t | x_{t-1}) p(x_{t-1} | z_{1:t-1}) dx_{t-1}$. The integral is “predict”, the multiplication is “update”.
5. **K-means termination.** Both steps are non-increasing in J (nearest centre; mean is the minimiser); finitely many assignments; monotone $\Rightarrow$ no assignment repeats $\Rightarrow$ halts. Local minimum only.
6. **Weiss median.** Sparse (Laplacian) prior on filtered illumination $\Rightarrow$ ML = minimise ℓ_1 error over time $\Rightarrow$ the estimator is the temporal *median*. Gaussian prior would have given the mean.

15 Last-look sheet

Which distance / which cost?

- Avoid edges (colour, road centre): $w = |\nabla I|$.
- Follow edges (scissors, hat brim): $w = 1/(|\nabla I| + \epsilon)$.
- Foreground-ness (matting): $w = |\nabla P(F | c)|$.
- Graph tie strength $\rightarrow$ length: $\ell = 1/w$ or $-\log w$.
- Point cloud: k -NN graph, $w = \|p - q\|$ (+ normal difference).

Which filter for tracking?

- Linear + Gaussian + unimodal $\Rightarrow$ Kalman.
- Non-linear / multi-modal / clutter $\Rightarrow$ particle filter.
- Sample by weight $\Rightarrow$ cumulative array + one `rand()`.

Data term vs smoothness term

- New *measurement* or confidence $\Rightarrow$ data term.
- New *boundary* signal $\Rightarrow$ smoothness weight.
- Both wanted $\Rightarrow$ do both, and say why.

Make it online / cheap

- Median or mode over time $\Rightarrow$ 256-bin histogram.
- Multi-modal per-pixel statistics $\Rightarrow$ incremental GMM.
- All-pairs sums $\Rightarrow$ low-rank kernel factorisation.
- Nearest-neighbour fields $\Rightarrow$ PatchMatch.
- Distance maps on a GPU $\Rightarrow$ fast sweeping.

The regularisation dial, everywhere

- HS α , super-res λ : data vs prior (ML vs MAP).
- Kalman R vs Q : sensor vs model.
- Colorization b : sharp vs soft blend.
- Retargeting α : completeness vs coherence.
- GMM T, α : what counts as background, how fast.

Assumptions that break (name them for free points)

- Brightness constancy: specularities, shadows, auto-exposure.
- LK: large motion, flat/edge windows (aperture problem).
- HS: quadratic smoothness blurs motion boundaries.

- Eulerian magnification: motion > 1 px, shaky camera.
- GMM: sudden lighting change, shadows, camera shake.
- Weiss: needs a fixed camera and sparse lighting.
- Dynamic texture: needs stationary statistics.
- PatchMatch: needs coherent correspondences; approximate.
- Retargeting: bad init (black) $\Rightarrow$ garbage; lines bend.

One-line summary of the course

Define a cost. Add a boundary-aware regulariser. Minimise it iteratively — by least squares, by Dijkstra/DP, or by alternating two steps that each only lower the cost.